

Angular Moderno

33 - Finalização Funcional do Projeto



Aula 33 — Finalização Funcional do Projeto

Deixar o sistema funcionando como um produto completo, garantindo persistência de dados, fluxo consistente entre telas e validação real do comportamento da aplicação.

Fluxo Completo

Catálogo → Carrinho → Checkout

Integração Final

Ajustes e correção de bugs

Validação Real

Comportamento de produto completo

Contexto da Aula

Até este ponto do curso, construímos todas as partes do sistema separadamente. Agora, o foco é **integrar tudo** e garantir que o sistema funcione como um produto real.

Carrinho

Funcione corretamente em todas as situações

Login

Seja mantido após atualização da página

Checkout

Finalize corretamente com estado consistente

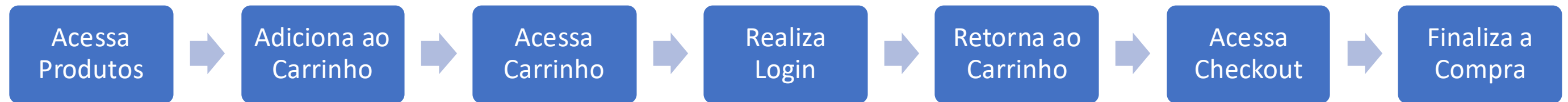
Produto Real

Comportamento esperado de aplicações modernas

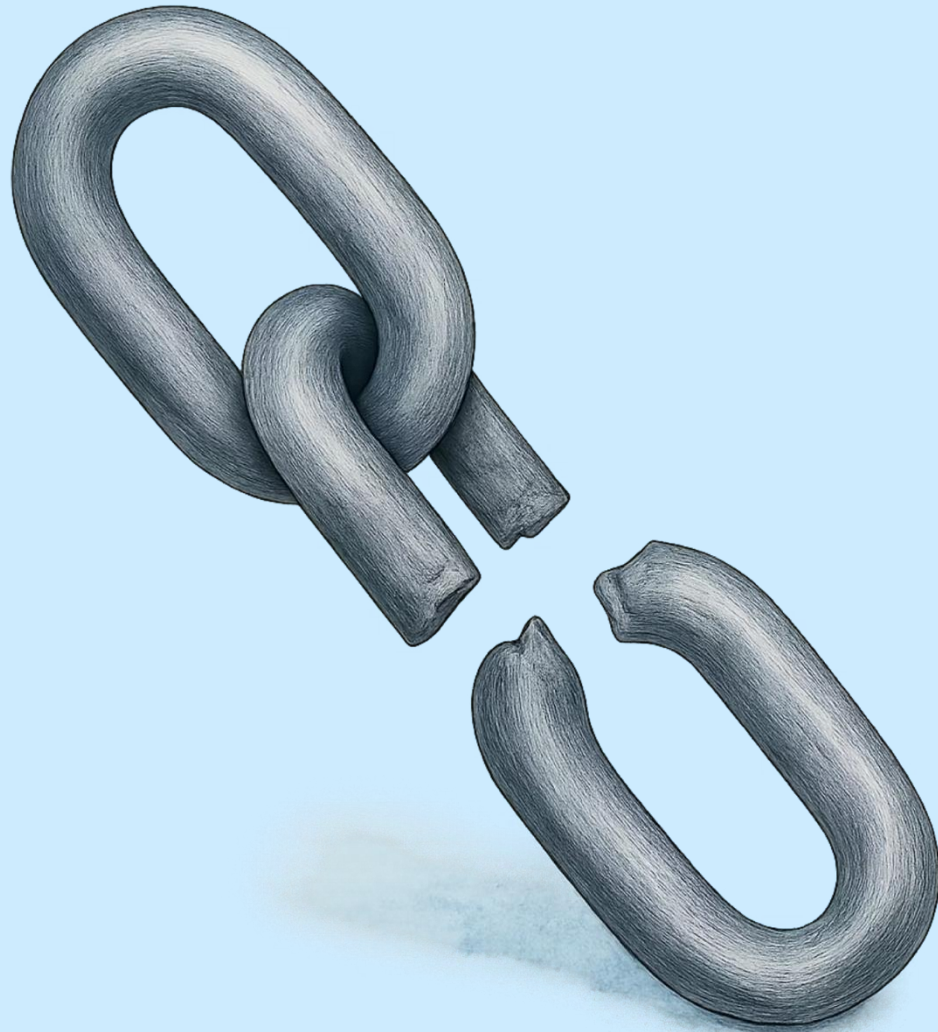


Fluxo Completo Esperado do Sistema

O sistema deve seguir um fluxo linear e consistente, do acesso inicial até a finalização da compra.



Cada etapa depende da anterior para funcionar corretamente. A rota do carrinho é protegida e exige autenticação antes de prosseguir.



Problema Identificado Antes da Aula

Antes desta aula, o sistema apresentava limitações críticas que impediam o funcionamento como produto real.

Carrinho Perdido

Itens desapareciam ao atualizar a página

Login não era restaurado após atualização da página

Sessão encerrada após reload da página

Checkout Inconsistente

Estado não era mantido corretamente

Sem Comportamento Real

Sistema não simulava uso real de aplicação

Solução Proposta Nesta Aula

Para resolver os problemas identificados, vamos implementar um conjunto de melhorias que transformarão o sistema em um produto funcional completo.



Persistência do Carrinho

Utilizar `localStorage` para manter itens entre recarregamentos



Persistência de Autenticação

Manter usuário logado após atualização da página



Ajustes no Checkout

Fluxo completo de finalização com confirmação de pedido



Validação Completa

Fluxo real de uso testado do início ao fim

Atenção

Antes de iniciar qualquer alteração estrutural no projeto, é necessário parar o servidor de desenvolvimento.



Ignorar este passo pode causar inconsistências durante alterações em serviços globais.

Etapa 1 — Parar o Servidor

No terminal do projeto, execute o atalho de teclado para interromper o servidor:

CTRL + C

Motivo: Evitar inconsistências durante alterações em serviços globais e estado da aplicação. Sempre pare o servidor antes de modificar arquivos de serviço críticos.

Etapa 2 — Persistência do Carrinho

Arquivo a ser alterado

```
src/app/core/services/carrinho.service.ts
```

Objetivo da Alteração

- Manter itens do carrinho mesmo após atualizar a página
- Utilizar `localStorage` como mecanismo de persistência
- Garantir compatibilidade com ambiente browser



Trecho a ser Alterado — `carrinho.service.ts`

APP/CORE/SERVICES/CARRINHO.SERVICE.TS

Realize as seguintes modificações no arquivo de serviço do carrinho:

1

Substituir inicialização do signal

```
private carrinho = signal<ItemCarrinho[]>(this.carregarCarrinhoSalvo());
```

2

Adicionar chave de storage

```
private readonly chaveStorage = 'minha-loja-carrinho';
```

3

Criar métodos auxiliares

- `carregarCarrinhoSalvo()`
- `salvarCarrinho()`
- `estaNoNavegador()`

Será necessário importar `isPlatformBrowser`, `PLATFORM_ID`, `effect` e `inject`.

Código Completo — carrinho.service.ts

APP/CORE/SERVICES/CARRINHO.SERVICE.TS

```
import { isPlatformBrowser } from '@angular/common';
import { Injectable, PLATFORM_ID, computed, effect, inject, signal } from '@angular/core';

import { ItemCarrinho } from '../models/item-carrinho';

@Injectable({
  providedIn: 'root',
})
export class CarrinhoService {
  // PLATFORM_ID permite verificar se o código está rodando no navegador.
  // Isso evita erro com localStorage em ambientes que não possuem browser.
  private platformId = inject(PLATFORM_ID);

  // Chave única usada para salvar e recuperar o carrinho no localStorage.
  private readonly chaveStorage = 'minha-loja-carrinho';

  // O carrinho inicia tentando recuperar os dados salvos no navegador.
  private carrinho = signal<ItemCarrinho[]>(this.carregarCarrinhoSalvo());

  itens = computed(() => this.carrinho());

  quantidade = computed(() => this.carrinho().length);

  total = computed(() =>
    this.carrinho().reduce((total, item) => total + item.preco, 0)
  );

  carrinhoVazio = computed(() => this.carrinho().length === 0);

  constructor() {
    // Sempre que o carrinho mudar, a lista atualizada será persistida.
    effect(() => {
      this.salvarCarrinho(this.carrinho());
    });
  }

  adicionar(produto: ItemCarrinho) {
    this.carrinho.update((listaAtual) => [...listaAtual, produto]);
  }
```

```
  removerPorIndice(indice: number) {
    this.carrinho.update((listaAtual) =>
      listaAtual.filter((_, index) => index !== indice)
    );
  }

  limpar() {
    this.carrinho.set([]);
  }

  // Método auxiliar para impedir uso de localStorage fora do navegador.
  private estaNoNavegador(): boolean {
    return isPlatformBrowser(this.platformId);
  }

  // Recupera o carrinho salvo, se existir.
  private carregarCarrinhoSalvo(): ItemCarrinho[] {
    if (!this.estaNoNavegador()) {
      return [];
    }

    const dadosSalvos = localStorage.getItem(this.chaveStorage);
    if (!dadosSalvos) {
      return [];
    }

    try {
      return JSON.parse(dadosSalvos) as ItemCarrinho[];
    } catch {
      return [];
    }
  }

  // Salva o carrinho atualizado no navegador.
  private salvarCarrinho(itens: ItemCarrinho[]) {
    if (!this.estaNoNavegador()) {
      return;
    }

    localStorage.setItem(this.chaveStorage, JSON.stringify(itens));
  }
}
```

Como Acontece a Persistência do Carrinho?

Com as alterações implementadas, o carrinho agora possui comportamento persistente entre sessões de navegação.

Ao Alterar o Carrinho

Os dados são automaticamente salvos no `localStorage` do navegador

Ao Recarregar a Página

Os dados são restaurados do `localStorage` automaticamente



Isso simula o comportamento real de aplicações modernas de e-commerce, onde o carrinho persiste entre visitas.



Etapa 3 — Persistência de Autenticação

Arquivo a ser alterado

```
src/app/core/services/auth.service.ts
```

Objetivo

- Manter usuário logado após atualização da página
- Simular sessão persistente com token JWT
- Restaurar estado de autenticação ao inicializar



Trecho a ser Alterado — `auth.service.ts`

APP/CORE/SERVICES/AUTH.SERVICE.TS

Realize as seguintes modificações no arquivo de serviço de autenticação:

1

Alterar inicialização dos signals

```
private usuario = signal<Usuario | null>(this.carregarUsuarioSalvo());  
private tokenJwt = signal<string | null>(this.carregarTokenSalvo());
```

2

Adicionar chaves de storage

```
private readonly chaveUsuario = 'minha-loja-usuario';  
private readonly chaveToken = 'minha-loja-token';
```

3

Criar métodos auxiliares

- `carregarUsuarioSalvo()`
- `carregarTokenSalvo()`
- `salvarAutenticacao()`
- `limparAutenticacaoSalva()`

Será necessário importar `isPlatformBrowser`, `PLATFORM_ID` e `inject`.

Código — auth.service.ts (Parte 1)

APP/CORE/SERVICES/AUTH.SERVICE.TS

```
import { isPlatformBrowser } from '@angular/common';
import { Injectable, PLATFORM_ID, computed, inject, signal } from '@angular/core';

type PerfilUsuario = 'usuario' | 'admin';

type Usuario = {
  email: string;
  perfil: PerfilUsuario;
};

@Injectable({
  providedIn: 'root',
})
export class AuthService {
  // Permite verificar se o código está rodando no navegador.
  private platformId = inject(PLATFORM_ID);

  // Chaves usadas para salvar autenticação simulada no localStorage.
  private readonly chaveUsuario = 'minha-loja-usuario';
  private readonly chaveToken = 'minha-loja-token';

  // O estado inicial tenta recuperar login salvo no navegador.
  private usuario = signal<Usuario | null>(this.carregarUsuarioSalvo());
  private tokenJwt = signal<string | null>(this.carregarTokenSalvo());

  usuarioAtual = computed(() => this.usuario());
  estaLogado = computed(() => this.usuario() !== null && this.tokenJwt() !== null);
  ehAdmin = computed(() => this.usuario()?.perfil === 'admin');
  token = computed(() => this.tokenJwt());

  login(email: string, senha: string): boolean {
    if (!email || !senha) {
      return false;
    }

    const perfil: PerfilUsuario = email === 'admin@email.com' ? 'admin' : 'usuario';

    const usuarioLogado: Usuario = {
      email,
      perfil,
    };
  }
```

```
const tokenSimulado =
  'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.' +
  'eyJzdWIiOiJhbHVub0B0ZXN0ZS5jb20iLCJwZXJmaWwiOiJlc3Vhcm1vIn0.' +
  'assinatura-simulada';

  this.usuario.set(usuarioLogado);
  this.tokenJwt.set(tokenSimulado);

  // Salva login simulado para manter a sessão após atualizar a página.
  this.salvarAutenticacao(usuarioLogado, tokenSimulado);

  return true;
}

logout() {
  this.usuario.set(null);
  this.tokenJwt.set(null);

  // Remove dados de autenticação ao sair.
  this.limparAutenticacaoSalva();
}

obterToken(): string | null {
  return this.tokenJwt();
}

obterPerfil(): PerfilUsuario | null {
  return this.usuario()?.perfil ?? null;
}

// Evita acessar localStorage fora do navegador.
private estaNoNavegador(): boolean {
  return isPlatformBrowser(this.platformId);
}

// Recupera usuário salvo.
private carregarUsuarioSalvo(): Usuario | null {
  if (!this.estaNoNavegador()) {
    return null;
  }
}
```

Código — `auth.service.ts` (Parte 2)

APP/CORE/SERVICES/AUTH.SERVICE.TS

```
const dadosSalvos = localStorage.getItem(this.chaveUsuario);

if (!dadosSalvos) {
  return null;
}

try {
  return JSON.parse(dadosSalvos) as Usuario;
} catch {
  return null;
}

// Recupera token salvo.
private carregarTokenSalvo(): string | null {
  if (!this.estaNoNavegador()) {
    return null;
  }
  return localStorage.getItem(this.chaveToken);
}

// Salva usuário e token simulados.
private salvarAutenticacao(usuario: Usuario, token: string) {
  if (!this.estaNoNavegador()) {
    return;
  }
  localStorage.setItem(this.chaveUsuario, JSON.stringify(usuario));
  localStorage.setItem(this.chaveToken, token);
}

// Limpa autenticação salva.
private limparAutenticacaoSalva() {
  if (!this.estaNoNavegador()) {
    return;
  }
  localStorage.removeItem(this.chaveUsuario);
  localStorage.removeItem(this.chaveToken);
}
}
```



Explicação da Persistência de Login

Com as alterações no serviço de autenticação, o sistema agora mantém a sessão do usuário de forma persistente.

Usuário Logado

Mantido após pressionar F5 ou
recarregar a página

Token JWT

Token simulado mantido no
`localStorage`

Navegação Contínua

Permite navegar entre páginas sem perder a sessão



Importante: Persistência de estado não significa controle de navegação.
São conceitos distintos.

Diferença Importante: Persistência vs Navegação

É fundamental entender a distinção entre esses dois conceitos para compreender o comportamento do sistema.

Persistência

Mantém dados no navegador entre recarregamentos.

- Salva no `localStorage`
- Restaura ao inicializar
- Independe de rotas

Navegação

Define para onde o usuário é redirecionado.

- Controlada por Guards
- Depende do estado de autenticação
- Pode redirecionar para login

 Mesmo logado, o sistema pode redirecionar para login dependendo da configuração da rota.

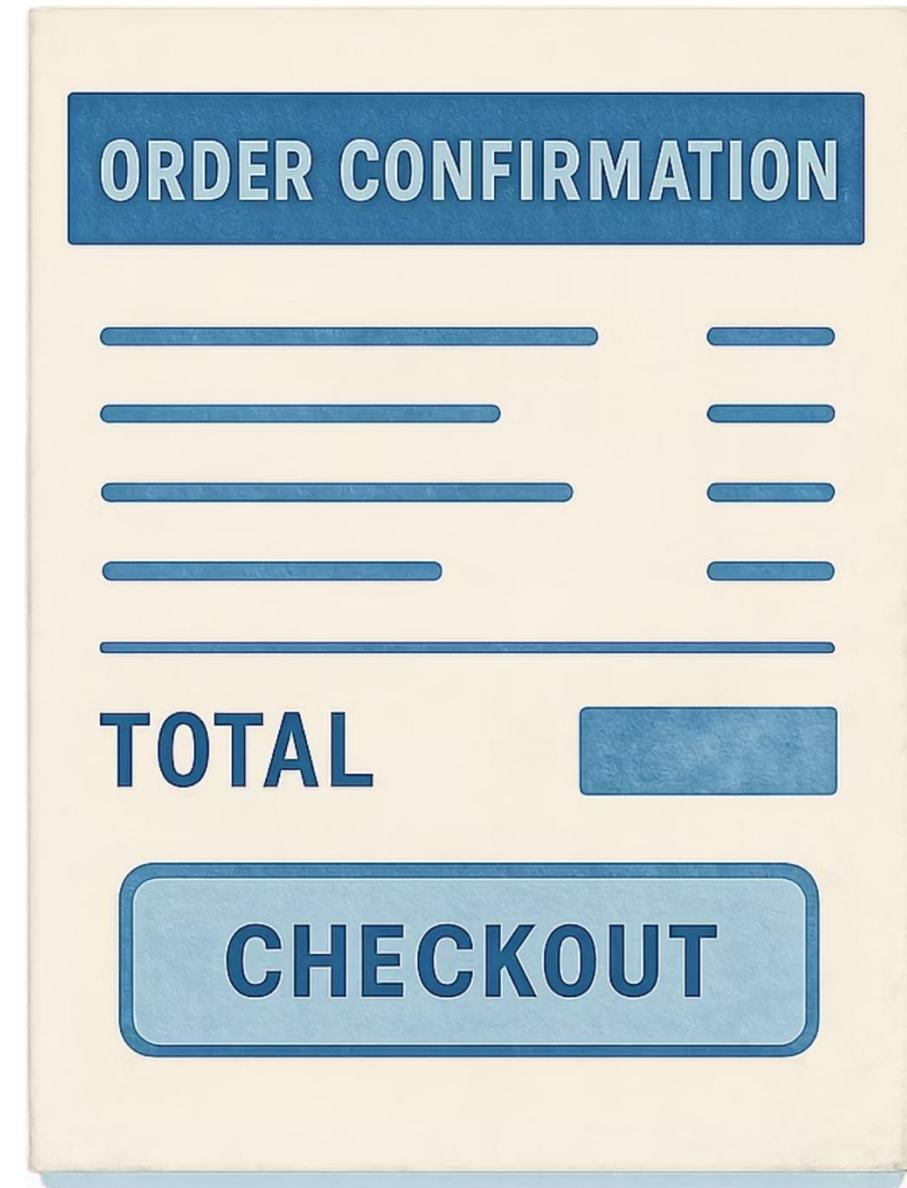
Etapa 4 — Ajustes no Checkout

Arquivo a ser alterado

```
src/app/features/checkout/checkout/checkout.ts
```

Objetivo

- Criar confirmação de pedido após finalização
- Exibir resumo com dados do pedido
- Limpar carrinho corretamente após compra
- Adicionar o tipo `PedidoFinalizado`
- Método `finalizar()` deve capturar os itens e o total antes de limpar o carrinho



Trecho a ser Alterado — `checkout.ts`

APP/FEATURES/CHECKOUT/CHECKOUT/CHECKOUT.TS

Realize as seguintes modificações no componente de checkout:

1

Adicionar signal de pedido finalizado

```
pedidoFinalizado = signal<PedidoFinalizado | null>(null);
```

2

Modificar método `finalizar()`

- Criar objeto de pedido com dados do formulário
- Armazenar dados do pedido no signal
- Limpar carrinho após finalização
- Resetar formulário para estado inicial

Código Completo — checkout.ts

APP/FEATURES/CHECKOUT/CHECKOUT/CHECKOUT.TS

```
import { Component, inject, signal } from '@angular/core';
import {
  AbstractControl,
  FormControl,
  FormGroup,
  ReactiveFormsModule,
  ValidationErrors,
  Validators,
} from '@angular/forms';
import { RouterLink } from '@angular/router';

import { CarrinhoFacade } from '../../../core/facades/carrinho.facade';
import { ItemCarrinho } from '../../../core/models/item-carrinho';

type PedidoFinalizado = {
  codigo: number;
  cliente: string;
  quantidadeItens: number;
  total: number;
  itens: ItemCarrinho[];
};

function nomeSemNumeros(control: AbstractControl): ValidationErrors | null {
  const valor = control.value;

  if (!valor) return null;

  if (/\\d/.test(valor)) {
    return { numeroInvalido: true };
  }

  return null;
}

@Component({
  selector: 'app-checkout',
  imports: [ReactiveFormsModule, RouterLink],
  templateUrl: './checkout.html',
  styleUrls: ['./checkout.css'],
})
```

```
export class Checkout {
  carrinhoFacade = inject(CarrinhoFacade);

  // Guarda os dados do pedido finalizado para exibir confirmação real na tela.
  pedidoFinalizado = signal<PedidoFinalizado | null>(null);

  formulario = new FormGroup({
    nome: new FormControl('', [Validators.required, Validators.minLength(3), nomeSemNumeros]),
    email: new FormControl('', [Validators.required, Validators.email]),
    endereco: new FormControl('', [Validators.required, Validators.minLength(5)]),
  });

  finalizar() {
    this.pedidoFinalizado.set(null);

    if (this.carrinhoFacade.carrinhoVazio()) {
      console.log('Não é possível finalizar uma compra com o carrinho vazio.');
      return;
    }

    if (this.formulario.invalid) {
      console.log('Formulário inválido');
      this.formulario.markAllAsTouched();
      return;
    }

    const dados = this.formulario.value;
    const itens = this.carrinhoFacade.itens();
    const total = this.carrinhoFacade.total();

    // Cria um resumo simples do pedido
    // antes de limpar o carrinho.
    const pedido: PedidoFinalizado = {
      codigo: Date.now(),
      cliente: dados.nome ?? '',
      quantidadeItens: itens.length,
      total,
      itens,
    };

    console.log('Compra finalizada com sucesso!');
    console.log('Pedido:', pedido);
    console.log('Dados do formulário:', dados);

    // Após finalizar, o carrinho global é limpo.
    this.carrinhoFacade.limparCarrinho();

    this.formulario.reset();
    this.pedidoFinalizado.set(pedido);
  }
}
```

Etapa 5 — Ajuste do Template do Checkout

Arquivo a ser alterado

```
src/app/features/checkout/checkout/checkout.html
```

Objetivo

- Exibir mensagem de sucesso após finalização
- Mostrar resumo completo do pedido
- Controlar exibição condicional com `@if`



Trecho a ser Alterado — `checkout.html`

APP/FEATURES/CHECKOUT/CHECKOUT/CHECKOUT.HTML

Adicione a estrutura condicional no template para exibir o resumo do pedido após a finalização:

```
@if (pedidoFinalizado()) {  
    // Exibir resumo do pedido  
}
```

Dentro do bloco condicional, exibir as seguintes informações:

Código do Pedido

Identificador único gerado para o pedido

Nome do Cliente

Nome informado no formulário de checkout

Total

Valor total da compra realizada

Quantidade

Número de itens incluídos no pedido

Este template também precisa controlar 3 estados: pedido finalizado, carrinho vazio e checkout em andamento.

Código Completo — checkout.html

APP/FEATURES/CHECKOUT/CHECKOUT/CHECKOUT.HTML

```
<h2>Checkout</h2>

@if (pedidoFinalizado()) {
  <section>
    <h3 style="color: green">Compra finalizada com sucesso!</h3>

    <p><strong>Código do pedido:</strong> {{ pedidoFinalizado()?.codigo }}</p>

    <p><strong>Cliente:</strong> {{ pedidoFinalizado()?.cliente }}</p>

    <p><strong>Itens comprados:</strong> {{ pedidoFinalizado()?.quantidadeItens }}</p>

    <p><strong>Total:</strong> R$ {{ pedidoFinalizado()?.total?.toFixed(2) }}</p>

    <button type="button" routerLink="/produtos">Fazer nova compra</button>
  </section>
}

@if (carrinhoFacade.carrinhoVazio() && !pedidoFinalizado()) {
  <p style="color: red">
    Seu carrinho está vazio. Adicione produtos antes de finalizar a compra.
  </p>

  <button type="button" routerLink="/produtos">
    Ver produtos
  </button>
} @else if (!pedidoFinalizado()) {
  <section>
    <h3>Resumo do pedido</h3>

    <p>Itens no carrinho: {{ carrinhoFacade.quantidade() }}</p>

    <h3>Total: R$ {{ carrinhoFacade.total().toFixed(2) }}</h3>

    <button type="button" routerLink="/carrinho">
      Revisar carrinho
    </button>
  </section>
  <hr />
}
```

```
<form [formGroup]="formulario">
  <p><input type="text" formControlName="nome" placeholder="Nome" /></p>
  @if (formulario.get('nome')?.touched && formulario.get('nome')?.invalid) {
    @if (formulario.get('nome')?.errors?.['required']) {
      <small>Nome é obrigatório</small>
    }
    @if (formulario.get('nome')?.errors?.['minlength']) {
      <small>Nome deve ter no mínimo 3 caracteres</small>
    }
    @if (formulario.get('nome')?.errors?.['numeroInvalido']) {
      <small>Nome não pode conter números</small>
    }
  }
  <p><input type="email" formControlName="email" placeholder="E-mail" /></p>
  @if (formulario.get('email')?.touched && formulario.get('email')?.invalid) {
    @if (formulario.get('email')?.errors?.['required']) {
      <small>Email é obrigatório</small>
    }
    @if (formulario.get('email')?.errors?.['email']) {
      <small>Email inválido</small>
    }
  }
  <p><input type="text" formControlName="endereco" placeholder="Endereço" /></p>
  @if (formulario.get('endereco')?.touched && formulario.get('endereco')?.invalid) {
    @if (formulario.get('endereco')?.errors?.['required']) {
      <small>Endereço é obrigatório</small>
    }
    @if (formulario.get('endereco')?.errors?.['minlength']) {
      <small>Endereço deve ter no mínimo 5 caracteres</small>
    }
  }
  <p>
    <button type="button" (click)="finalizar()" [disabled]="formulario.invalid">
      Finalizar Compra
    </button>
  </p>
</form>
}
```

Comando para iniciar

```
ng serve
```

Acessar no navegador

```
http://localhost:4200
```

Etapa 6 — Retomar o Servidor

Com todas as alterações implementadas, é hora de retomar o servidor de desenvolvimento e verificar o funcionamento do sistema.

- ✔ Após executar `ng serve`, aguarde a compilação completa antes de acessar a aplicação no navegador.

Roteiro de Testes – Parte 1

Inicie o roteiro de testes verificando o funcionamento básico do catálogo de produtos e do contador do carrinho.

01

Acessar a página de produtos

Navegue até a rota `/produtos` no navegador

03

Verificar contador no header

O número exibido no ícone do carrinho deve refletir a quantidade adicionada

02

Adicionar 2 produtos ao carrinho

Clique no botão de adicionar para dois produtos diferentes



Roteiro de Testes — Parte 2

01

Acessar a rota do carrinho

Navegue até `/carrinho` no navegador

Resultado Esperado

O sistema deve redirecionar automaticamente para a página de **login**, pois a rota `/carrinho` é protegida e exige autenticação.

 Este comportamento é controlado pelo Guard de rota, não pela persistência de dados.

Roteiro de Testes — Parte 3

Realize o login com as credenciais de teste para acessar as rotas protegidas.

Email

`usuario@email.com`

Senha

`123`

01

Fazer login com as credenciais acima

Preencha o formulário e confirme o acesso

02

Retornar ao carrinho

Navegue novamente para `/carrinho` após autenticação

Roteiro de Testes — Parte 4

01

Pressionar F5

Recarregue a página do carrinho com o atalho de teclado

Se voltar para `/login` após pressionar F5, acesse novamente `/carrinho` para confirmar que os itens continuam salvos.

Resultado Esperado

Os itens devem **continuar no carrinho** após o recarregamento, graças à persistência com `localStorage`.

Observação

Se o sistema redirecionar para login após F5, isso é **comportamento de navegação** (Guard), não falha na persistência de dados.

Roteiro de Testes — Parte 5

Teste o fluxo completo de finalização de compra no checkout.

01

Acessar o checkout

Navegue até a rota `/checkout`

02

Preencher o formulário

Complete todos os campos obrigatórios do formulário de checkout

03

Finalizar a compra

Clique no botão de finalização e aguarde a confirmação

Roteiro de Testes — Parte 6

01

Verificar resumo do pedido

Confirme que o código, nome, total e quantidade são exibidos corretamente

02

Confirmar carrinho vazio

Verifique que o carrinho foi limpo após a finalização da compra

ORDER SUMMARY

Total **\$25:20**

CONFIRMATION

Roteiro de Testes — Parte 7

01

Pressionar F5

Recarregue a página após a finalização do pedido

Caso o navegador volte para /login após F5, isso não indica perda da sessão; é comportamento de navegação. Verifique se o header ainda reconhece o usuário logado ou tente acessar novamente uma rota protegida.

Resultado Esperado

O **usuário deve continuar logado** após o recarregamento da página, graças à persistência de autenticação implementada no `auth.service.ts`.



Este é o comportamento esperado de aplicações modernas com sessão persistente.

Roteiro de Testes — Parte 8

01

Realizar logout

Clique no botão de logout para encerrar a sessão

Resultado Esperado

Após o logout, o acesso a **rotas protegidas deve ser bloqueado**. Tentativas de acessar `/carrinho` ou `/checkout` devem redirecionar para o login.



Confirme que os dados de autenticação foram removidos do `localStorage`.

Conclusão da Aula

Parabéns! O sistema agora funciona como um produto acadêmico real, com todas as funcionalidades integradas e operando de forma consistente.



Carrinho Persistente

Itens mantidos entre recarregamentos



Login Ativo

Sessão mantida após F5



Checkout Completo

Fluxo de compra funcional



Produto Real

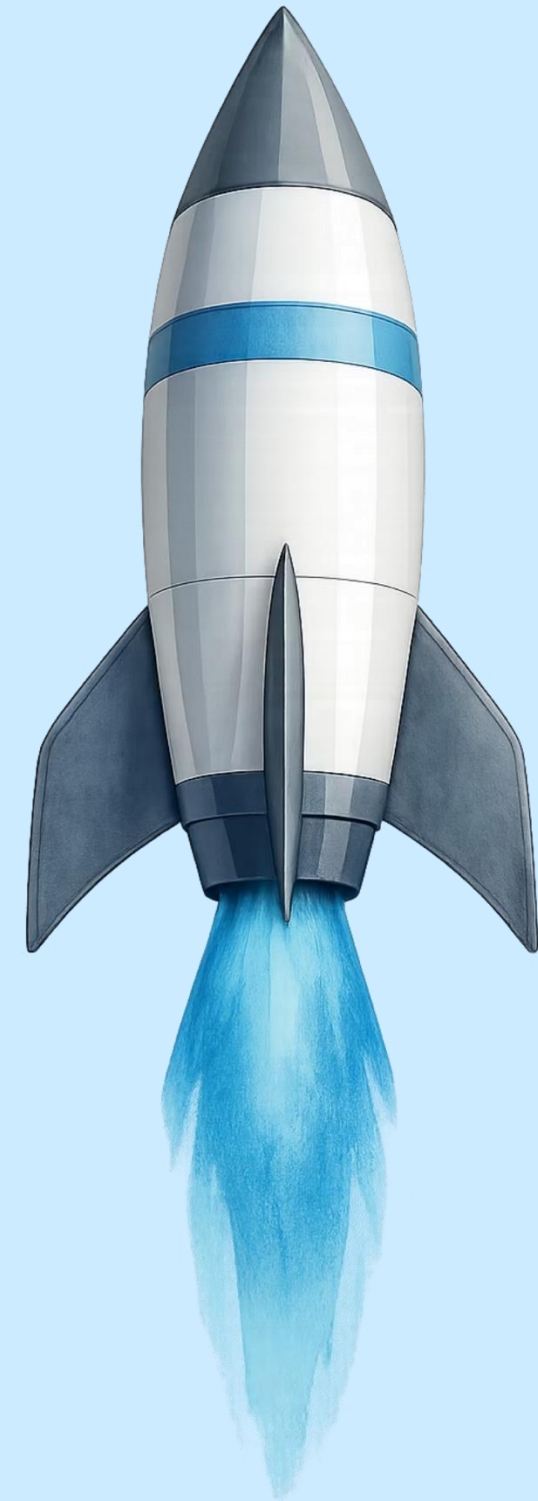
Comportamento de aplicação moderna



Evolução Futura do Projeto

O sistema está funcional como produto acadêmico. A seguir, temos sugestões para melhorias futuras que podem elevar o projeto a um nível ainda mais próximo de aplicações de produção.

- 1 Redirecionamento Inteligente
Após login, redirecionar para a rota original solicitada
- 2 Backend Real
Integração com API REST substituindo dados simulados
- 3 Banco de Dados
Persistência real em servidor, não apenas no navegador
- 4 Histórico de Pedidos
Consulta e acompanhamento de compras anteriores



Dúvidas

?

